

wll_OS

BuildStorm 内核设计优化文档

全国大学生计算机系统能力大赛操作系统内核实现赛·决赛

参赛学校

华南师范大学

队伍名称

12345

队伍编号

T2026105749910208

项目成员

吴秉谦 李骏东 罗锦程

2026 年 8 月 17 日

摘要

BuildStorm 以完整 Rust 工具链和多 crate 从零编译为负载，同时覆盖动态链接、进程创建与回收、SMP 调度、虚拟内存、页表与 TLB、文件系统命名空间以及缓存写回。它不是某一条系统调用的功能测试，而是一条会持续放大内核正确性缺口和长路径开销的综合压力链。

本项目从“脚本已找到但第一条命令未执行”出发，先修正内核地址空间的使用边界和 Linux ABI 缺口，使工具链能够真实运行；随后围绕用户页所有权、COW 与进程生命周期建立清晰分层，解决低地址映射空间耗尽、文件系统命名空间竞态和大文件读取错误；在此基础上，通过有界诊断定位到中央堆锁、VFS 重复查询、空闲核唤醒风暴及无效内存回收等瓶颈，再逐项进行单变量验证。

最终候选在未修改的官方镜像和脚本、QEMU 11.0.3、`-snapshot -m 8G -smp 8`、production 内核和官方 judge 下完成双架构全量编译。RISC-V64 与 LoongArch64 的最终公开流程用时分别为 686.50 s 和 558.27 s，串口均包含精确成功标记，官方脚本自动项均为 180/180。

关键词： BuildStorm；内核设计；虚拟内存；SMP；ext4；VFS；页帧所有权；性能归因；双架构验证

评分要求对应关系

评委快速核对入口

评审内容	主要位置
问题（bug 或性能瓶颈）定位与根因分析	第 2 章
修复或优化的设计与实现	第 3、4 章
修改前后的编译时间、加速比等数据对比	第 5 章
AI 使用说明与可复现开发过程	第 6—8 章

表 0-1 评分项与正文位置

0.1 证据口径

本文使用三种证据等级。**official-pass** 表示未修改官方镜像和脚本，使用规定资源、production 内核及官方 judge，并出现精确成功标记；**capability-pass** 表示独立 ABI、SMP、生命周期或不同资源配置下的能力回归通过；**unverified** 表示短窗口、诊断版本、派生镜像、不同配置或缺少精确成功标记。所有“完成”和“加速”结论都以第一种证据为上限，局部计数器只用于归因。

目录

摘要	I
评分要求对应关系	II
0.1 证据口径	II
目录	III
第 1 章 任务理解与总体结论	1
1.1 综合负载带来的内核压力	1
1.2 从不可运行到稳定完成	1
1.3 最终结果与诚实边界	1
第 2 章 问题定位与根因分析	2
2.1 定位原则：先找最早真实边界	2
2.2 地址空间边界与 Linux ABI	2
2.2.1 syscall 内核执行边界	2
2.2.2 工具链所需的通用语义	2
2.3 内存所有权、生命周期与地址容量	3
2.4 文件系统：一致性问题先于缓存问题	3
2.5 并行与分配器：高 CPU 不等于高吞吐	3
2.6 大资源拓扑与隐藏阶段	3
第 3 章 总体架构与设计不变量	4
3.1 围绕所有权与发布顺序分层	4
3.2 内存四层职责	4
3.3 进程生命周期与可观察事件	4
3.4 调度与唤醒协议	4
3.5 文件系统控制面与数据面	5
3.6 跨架构边界	5
第 4 章 关键修复与优化实现	6
4.1 用户内存：从表示修复到回收语义	6
4.1.1 稀疏驻留集与固定页帧引用	6
4.1.2 高地址 mmap arena	6
4.1.3 MADV_DONTNEED 的真正回收	6
4.2 内核堆：IRQ-safe 的 per-CPU 小对象缓存	6
4.3 调度与定时器	6
4.4 ext4/VFS：正确性与吞吐的连续链条	7
4.4.1 大文件读取和命名空间事务	7
4.4.2 稀疏大文件缓存与聚簇写回	7
4.4.3 lookup 复用与有界工作集	7
4.5 ABI 与隐藏流程兼容	7
第 5 章 优化实验与结果分析	8
5.1 阶段数据的正确读法	8
5.2 中央堆优化的单变量证据	8

5.3	VFS 查询复用的同配置 A/B	8
5.4	正确性和资源效率改动如何评价	9
5.5	被证伪候选与取舍	9
5.6	与本地 judge Linux 基线的比较	9
第 6 章	测试体系与回归验证	10
6.1	验证矩阵	10
6.2	独立回归覆盖的关键情形	10
6.3	正式成功条件	10
6.4	生产与诊断隔离	10
第 7 章	可复现步骤与证据身份	11
7.1	权威输入	11
7.2	更新并核验官方测例	11
7.3	静态检查与双架构构建	11
7.4	独立 SMP 回归	12
7.5	官方完整运行与 judge	12
7.6	证据索引	12
第 8 章	AI 使用说明与人工核验	13
8.1	AI 参与的工作	13
8.2	人工负责的判断与检查	13
8.3	未由 AI 或运行器改变的内容	13
8.4	复核开发过程的最短路径	13
第 9 章	限制、后续工作与总结	14
9.1	当前限制	14
9.2	后续方向	14
9.3	总结	14

第 1 章 任务理解与总体结论

1.1 综合负载带来的内核压力

BuildStorm 的外层目标是完成 `tgskits` 的 `clean build`，但对内核而言，它更像一条长时间、多阶段的系统能力组合。工具链启动阶段首先要求 ELF 装载、动态链接、信号和大文件读取正确；`Cargo` 展开依赖后，短命子进程、管道、`Unix socket`、文件锁与等待队列会被反复调用；多个 `rustc` 并行时，虚拟地址碎片、匿名页、`COW`、页表更新和跨核失效成为持续压力；海量源码和中间文件又会放大目录一致性、`inode`/页缓存容量以及写回粒度。

因此，我们没有把“为某个 `crate` 增加特殊路径”作为解决方向。整个开发过程只接受能够表述为通用内核不变量的修改：地址空间何时激活、对象由谁拥有、状态何时发布、缓存何时失效，以及平台相关操作由哪一层完成。这样的修改可以由独立回归验证，也能同时解释 `RISC-V64` 与 `LoongArch64` 的行为。

1.2 从不可运行到稳定完成

项目经历了三个性质不同的阶段。第一阶段解决“能否运行”：脚本最初在进入第一条命令前失败，随后又暴露出 `IPC`、文件锁、子进程等待及大型 ELF 读取缺口。第二阶段解决“能否长期正确运行”：用户页所有权、`vfork/COW`、映射空间容量和 `ext4` 命名空间竞态在短测试中并不明显，却会在完整编译中导致超时、内存错误或中间文件 `ENOENT`。第三阶段才进入“是否高效”：诊断数据将主要开销收敛到中央内核堆、`VFS` 重复解析、无效空闲核唤醒以及不完整的内存回收语义。

最终保留的实现既包含性能优化，也包含没有明显缩短用时但必须存在的正确性修复。例如，命名空间事务只使 `900 s` 窗口的编译事件从 129 增至 131，却消除了中间文件发布竞态；单空闲核唤醒几乎不改变 `crate` 边界，但把无用 `QEMU CPU` 消耗从约 720% 降到约 107%。本文会分别说明其价值，不用同一把“加速比”衡量所有改动。

1.3 最终结果与诚实边界

架构	资源	编译时间	产物大小	judge	证据
RISC-V64	8 GiB / 8 vCPU	686.50 s	1,683,456 B	180/180	official-pass
LoongArch64	8 GiB / 8 vCPU	558.27 s	1,716,224 B	180/180	official-pass

表 1-1 最终公开流程结果

两次运行均使用未修改官方镜像、原始来宾脚本、`snapshot` 模式和 `diagnostics` 关闭的 `production` 内核，串口包含 `BUILDSTORM_COMPILE mode=multi ok=true`。`LoongArch judge` 保留了其版本中的旧核数警告，但自动项仍完整通过；本文不据此推断评测机最终时间分。评测机 `Linux` 基线会在同机重测，隐藏 `UEFI/nested-QEMU` 阶段也不能由公开脚本代替，因此相关状态在新提交前仍需单独确认。

第 2 章 问题定位与根因分析

2.1 定位原则：先找最早真实边界

长时间编译很容易产生“还在跑就是慢”“某个 crate 停留就是该 crate 有问题”的错觉。我们的诊断从最早可重复边界开始：保存完整串口和 QEMU 参数，以官方 marker 区分工具链、minibuild 与全量编译；把失败翻译为通用内核能力，再用独立回归验证；只有前置正确性成立后，才在 production 对照中一次改变一个假设。

诊断代码与生产代码严格分离。计数器只做聚合，不输出每次 syscall 或每个路径；锁等待、命名空间代际、页帧所有权影子状态和 QMP 地址归因均由显式 feature 控制。这样既降低观测扰动，也避免生产内核根据测试名称、crate、路径、命令、输出或 marker 改变行为。

现象	关键证据	根因	影响层次
脚本找到但第一条输出未出现	VFS/VirtIO 操作发生时已恢复用户根页表	syscall 期间地址空间切换过早	执行边界
Cargo 子进程无法收敛	阻塞集中在管道、socket、锁和 child wait	Linux IPC 与等待语义不完整	ABI/进程
LA 大型 PIE 重定位异常	ELF 头正确、约 14 MiB 后部内容错块	ext4 多层 extent 后部解析不完整	文件读取
长跑 mmap 大量 ENOMEM	最大空闲间隙约 85.37 MiB，却请求 128 MiB 懒映射	低地址 arena 容量与碎片化	虚拟内存
rustc 元数据偶发 ENOENT	lookup 跨越 writer 代际并重新发布旧缓存	命名空间缺少完整事务边界	文件系统一致性
多核高 CPU 但有效工作少	七个空闲 vCPU 各约 88% CPU、频繁自愿切换	每次入队广播 IPI，形成唤醒风暴	调度/SMP
中央堆反复自旋	3374 万次锁获取、QMP 聚集于 buddy 合并	小对象全部进入中央锁并触发 split/merge	内核分配器
jemalloc 报回收无效	135 次 fallback；discard 后读回旧字节	MADV_DONTNEED 只返回成功而未回收	内存回收 ABI

表 2-1 关键现象、证据与根因

2.2 地址空间边界与 Linux ABI

2.2.1 syscall 内核执行边界

最初的表面现象像是 shell 没有执行，实际问题发生在更低层：syscall 包装层在 VFS 和 VirtIO 工作结束前恢复了用户页表。用户低地址映射可能覆盖 QEMU virt 平台的 MMIO 区间，导致随后块设备访问在错误地址空间中进行。修复后的规则非常简单：syscall、VFS、设备和调度器内部始终使用内核地址空间，只有即将真正返回用户态时才激活目标进程地址空间。

这条规则把共享策略与平台动作分开。通用调度层决定“何时进入用户地址空间”，架构层只负责页表根寄存器、ASID 和 TLB 操作。它既解释了第一条命令之前的失败，也为后续 exec、阻塞和跨核迁移建立了稳定边界。

2.2.2 工具链所需的通用语义

页表边界修复后，rustup、rustc 与 Cargo 已能启动，但子进程仍可能永久等待。阻塞快照最终收敛到四类通用能力：描述符非阻塞状态和可读字节查询、保留消息边界的本地 seqpacket、对端关闭后的 EOF，以及按 open-file-description 持有的文件锁。另一个独立竞态出现在 child exit 与 wait 之间：父进程先检查状态、后登记睡眠时，子进程可能恰好退出并唤醒空队列。

这些修复均按 Linux 可观察语义实现。dup/fork/close 共享同一打开文件描述；seqpacket 不退化成字节流；等待路径采用“先登记、再复查条件”的原子边界。实现没有检查 Cargo 或 rustc 名称，因此同一能力也由独立 IPC 和生命周期回归覆盖。

2.3 内存所有权、生命周期与地址容量

长跑问题的共同特征，是旧设计容易把“虚拟区间策略”“已经驻留的物理页”和“硬件页表条目”混成一个对象。`fork`、`exec`、`exit`、共享映射和文件映射交错后，析构可能扫描大量空洞，COW 状态也可能与 PTE 权限不同步。诊断进一步显示，重复晚期停顿并非先由物理内存耗尽引起，而是低地址空间碎片化使 128 MiB 懒映射无处放置。

根因由两个层次组成。第一，所有权表示不清晰会放大 `exec` 和退出的析构成本，并增加重复释放风险；第二，即使物理页按需分配，VMA 本身仍需要连续虚拟地址，单一低区无法容纳编译器长期形成的映射拓扑。因此修复不能只扩大堆或调节扫描阈值，必须同时建立页级所有权边界和独立的高地址映射区。

`vfork` 和嵌套 COW 又验证了这一判断。父进程若在子进程 `exec/exit` 前恢复，会重用共享栈并覆盖参数；已经发生一次 COW 的页面若在下一次 `fork` 时只看 VMA 标志，可能留下仍可写的共享 PTE。最终实现分别用发布—唤醒握手和页级状态迁移解决，而不是依靠 BuildStorm 的进程顺序。

2.4 文件系统：一致性问题先于缓存问题

LoongArch64 大型 PIE 的头部可以读取，后部动态表却被错误解释。只读提取证明文件本身未损坏，错误来自旧 `ext4` 读取路径未完整遍历多层 `extent tree`。该修复使任意大文件都走 `extent-aware` 读取，而不是按 `rustc` 路径选择实现。

更隐蔽的问题发生在命名空间。900 s 运行中，`rustc` 在生成元数据时偶发 `ENOENT`。诊断记录到 146 次 `positive crossing` 和 170 次 `negative crossing`：reader 在 writer 修改前开始解析，却可能在 writer 失效缓存后把旧结果重新发布。静态审计也发现 `rename/unlink` 的父目录解析没有被同一外层锁覆盖。由此可知，根因不是某一次缓存删除遗漏，而是“解析—修改—失效—发布”没有形成完整事务。

一致性修复后，VFS 性能热点才有解释意义。完整诊断显示一次 `open` 会在同一命名空间代际内重复查询目录、普通文件和最终 `inode` 类型；16K `parent/readlink cache` 满载时清空整表，使大于缓存容量的工作集周期性变冷。这两项分别对应“同一结果未复用”和“淘汰形成性能悬崖”，不能用放宽一致性来换取速度。

2.5 并行与分配器：高 CPU 不等于高吞吐

宿主线程采样显示，一个 vCPU 接近满载时，其余空闲 vCPU 也各占用约 88% CPU，并产生每秒数万次自愿切换。来宾计数器只看到 `ready queue` 活动，无法直接表现 TCG 对广播 IPI 的放大。根因是本地时间片重排队与真正的外部 `runnable` 发布没有区分，每次入队都唤醒所有核。

另一个稳定热点来自内核堆。BuildStorm 会产生大量 8 B 至 4 KiB 的短命内核对象，旧中央 `buddy` 让每次分配和释放都获取同一把锁，释放时还可能跨多个 `order` 合并。900 s `diagnostics` 记录到 33,741,372 次中央锁获取和 5,004 次竞争，QMP 样本反复落在释放合并与自旋位置。这里的因果预测很明确：若小对象批量留在 CPU 本地，中央交互次数和自旋样本应同时大幅下降；后续数据验证了这一点。

2.6 大资源拓扑与隐藏阶段

评测资源扩大到 LoongArch64 36 GiB/12 CPU 后，扁平页帧引用表需要一次申请约 72 MiB，`buddy` 向上取整为 128 MiB，而动态堆尚未扩展；同时，十二个 128 KiB 启动栈共需 1.5 MiB，旧汇编仅保留 1 MiB，后四个 CPU 会越界破坏相邻 BSS。这两个问题都来自规模假设，不是编译负载特判。

RISC-V64 的 `nested-QEMU` 暴露同步 `SIGILL` 兼容性。QEMU 安装扩展探测 `handler` 后主动执行候选指令，旧内核却直接终止进程，导致运行日志为空。修复后，可捕获且未屏蔽的异常通过通用信号框架交给 `handler`；其他同步故障终止，避免在同一 PC 活锁。

第 3 章 总体架构与设计不变量

3.1 围绕所有权与发布顺序分层

最终架构没有把 BuildStorm 当作新的内核子系统，而是沿现有边界收紧职责。用户可见策略位于通用进程、内存和 VFS 层；物理页、打开文件描述和 inode 各有唯一所有权模型；页表只表达硬件映射；RISC-V64 与 LoongArch64 平台层实现激活、ASID、TLB 和中断运输。性能优化必须留在其责任单元内，不能通过跨层缓存或绕过失效协议获得短期收益。

用户工具链与编译负载
Linux ABI: 进程 / IPC / 信号 / 文件描述符 / 内存映射
通用内核服务: 调度与等待 · VMA 与驻留页 · VFS 与 ext4
资源所有权: 页帧引用 · 打开文件描述 · inode 与缓存失效
平台边界: RISC-V64 / LoongArch64 页表、TLB、IPI、启动与设备

图 3-1 BuildStorm 相关内核分层

3.2 内存四层职责

内存子系统被拆为四个相互约束的层次。VMA 描述虚拟区间、权限和 backing 策略；稀疏驻留集只持有实际出现的页面，并以连续 run 降低大空洞的管理成本；页帧跟踪器通过固定原子引用计数表达共享和最后释放；页表只维护虚拟页到物理页的硬件映射，不取得用户数据页的生命周期所有权。架构层在这四层之外负责 root 激活、ASID 分配以及本地/远端 TLB 失效。

这套分层使操作顺序可以被明确表述。例如，撤销一段私有匿名驻留页时，先从驻留集中提取 owner，再撤销页表 leaf，发布写入并完成本地 flush 与远端 shutdown，最后释放 owner。若先释放物理页，远端 CPU 可能仍通过旧 TLB 访问已经再分配的数据；若页表拥有物理页，又会与 COW 引用计数形成双重所有权。

3.3 进程生命周期与可观察事件

fork 将 private writable 映射转为 COW，只读映射直接共享，shared 映射保持共享语义。exec 先完整构建新地址空间，成功后一次替换，再按“执行身份—页表—驻留 owner”的顺序退休旧状态；失败时旧进程映像保持可用。exit 先从可调度结构中发布终止状态，等待者通过统一事件观察，再释放资源。vfork 则增加单独的父子握手，保证父进程只在子进程 exec 成功或退出后恢复。

等待队列采用事件计数和条件复查组合。阻塞方先登记自己，再检查目标条件；发布方先改变可观察状态，再唤醒。这条顺序被 pipe、socket、futex 和 child wait 复用，避免每个子系统分别实现容易丢事件的睡眠协议。

3.4 调度与唤醒协议

用户任务和内核任务的 runnable 队列保持顺序，并以 PID 集合防止重复发布。空闲 CPU 在最后一次检查队列之前先发布 idle bit；外部 producer 先把任务放入队列，再从 idle mask 中原子领取一个满足 affinity 的 CPU 并发送一次 IPI。于是只有两种结果：空闲侧复查看到任务，或 producer 领取 bit 并打断它，不存在任务已发布但所有 CPU 永久休眠的间隙。

本地时间片结束、syscall 返回和内核任务重排队不再广播唤醒；fork、阻塞完成、vfork 释放和新任务属于外部发布。可能同时影响多任务的 affinity 或线程组控制仍允许广播。该区分减少虚拟化环境中的唤醒群，同时不改变任务优先级和亲和性语义。

3.5 文件系统控制面与数据面

文件系统设计将命名空间控制面和普通文件数据面分离。控制面使用一个外层读写事务：lookup 从接受缓存、遍历 ext4 到发布 positive/negative 结果都持共享锁；create、link、unlink、rename 与 exchange 从第一次解析到落盘修改、缓存失效和 generation 更新都持独占锁。writer 先占据可升级槽，阻止新 reader 涌入，避免自旋读写锁中的 writer 饥饿。

普通数据读写不持命名空间事务锁。小文件保留紧凑缓存，大文件采用按页稀疏缓存；脏数据按 256 KiB 范围处理，连续完整块最多以 64 块聚合提交，尾部不完整块继续执行读—改—写。inode metadata、clean page、物理块连续 run 和 executable image 分层缓存，各自拥有明确容量和失效入口。这样既避免整文件 Vec 占用，又不让数据缓存夺取 inode 或页帧的生命周期所有权。

3.6 跨架构边界

共享内存管理层只提出“激活某地址空间”“撤销某段映射”“使某 root 的远端翻译失效”等请求。RISC-V64 根据 SATP/ASID 和 SBI RFENCE 完成动作；LoongArch64 使用其 PTE 格式、root CSR 与 IOCSR IPI/确认路径。平台层的保守策略可以不同，但上层 VMA、驻留页、COW、文件映射和缓存语义必须一致。

启动容量也遵循同一原则：页帧引用元数据按已发现内存区域分块初始化，两种架构共享计数和所有权逻辑；汇编入口各自保留足够的启动栈，Rust 在唤醒次级 CPU 前用链接符号检查实际空间，避免配置值与汇编常量再次漂移。

范围	不变量	独立验证
地址空间	内核工作不在错误用户根下执行；平台层独占激活与 TLB 语义	双架构 ASID/TLB 回归
页帧	共享只由页级引用计数表达；撤销翻译完成后才释放最后 owner	resident/COW/共享映射回归
进程	exec 原子替换；vfork 父进程等待；exit 先发布再回收	nested COW 与 child wait 回归
调度	任务先入队再唤醒；idle bit 先发布再复查；重复入队不发 IPI	八 CPU dispatch 与 affinity 回归
命名空间	解析、修改、失效与结果发布位于同一事务代际	七 reader/一 writer 压力回归
缓存	容量与复用不能绕过 generation、精确失效和对象生命周期	rename/open-unlink/fsync/truncate
内核堆	CPU-local 与中央锁不同时持有；IRQ 状态成对保存恢复	160/320 MiB heap stress

表 3-1 最终实现必须维持的核心不变量

第 4 章 关键修复与优化实现

4.1 用户内存：从表示修复到回收语义

4.1.1 稀疏驻留集与固定页帧引用

驻留页不再跟随整个 VMA 长度建立密集对象，而是按实际出现的连续页段记录。区间拆分、提取和析构只访问相关 run，不扫描尚未 fault 的大块地址。页帧 owner 只保存物理页号，clone/drop 在预分配引用表中执行原子增减；最后一个 owner 才把页面归还 buddy。页表页只能由“唯一 owner”显式转为 raw ownership，共享页禁止转换，避免用户页、页表页与连续 DMA 范围交叉释放。

为了支持 36 GiB 资源，引用计数由机器字收紧为 AtomicU32，并按最多 2^{18} 个 frame 分块。每个 production 块约 1 MiB，不再要求早期 buddy 提供 128 MiB 连续阶。diagnostics 的 owner shadow 也独立分块，但完全不进入 production。

4.1.2 高地址 mmap arena

历史低地址区继续服务固定映射和兼容负载；当低区无法满足无提示映射时，分配器转向两个架构均合法的正 canonical 高地址区。两个 arena 分别维护游标，映射仍为懒分配，因此扩大的是虚拟容量而不是立即占用物理内存。固定地址、共享内存和文件映射统一通过合法区间策略校验，不允许绕开用户上限或侵入共享内核根。

该方案针对“连续虚拟空间不足”而不是“总物理内存不足”。诊断中，高 arena 启用后记录到 5,542 次成功选择且 selection ENOMEM 为 0；1,800 s production 首次越过长期停留的第 33 个编译事件。

4.1.3 MADV_DONTNEED 的真正回收

旧实现对 syscall 233 直接返回成功，jemalloc 写入后请求丢弃，再读取时仍得到旧字节，因此回退到同步 memset。最终实现只处理完整覆盖范围内的 private anonymous resident；匿名共享、SysV shm、共享/私有文件映射均不误丢弃。VMA 拓扑保持不变，后续 fault 按原策略获得新的零页。

页表撤销以 2 MiB leaf-table 边界复用 walk，随后执行发布 fence、本地 flush 和以地址空间 root 为键的远端 shutdown，最后释放暂存 owner。独立回归验证写入非零内容、discard、翻译消失、refault 后全零，并确认 VMA 数量未变化及 anonymous shared 页面仍保持共享。

4.2 内核堆：IRQ-safe 的 per-CPU 小对象缓存

中央 buddy 仍是底层 heap range、大对象、动态扩展和最终 OOM 判断的唯一管理者；其上增加 8 B 至 4 KiB 的标准 size class。每 CPU、每 class 最多缓存 64 个 block，miss 最多批量补充 16 个，满载时批量回收 16 个。CPU-local 命中只获取当前 CPU 的小锁；跨 CPU free 进入执行释放动作的当前 CPU cache，不维护对象的原 CPU 归属。

实现中最重要的不是 size class 数量，而是中断和锁顺序。第一次候选在进入中央 allocator 时没有禁止本地中断，timer 或 VirtIO 中断可能在同一 CPU 递归获取 cache/中央锁。最终采用本地 IRQ 状态保存—关闭—恢复 guard：它在临界期固定 CPU 身份，并阻止同 CPU 递归；cache 锁与中央锁从不同时持有。中央分配失败时会 drain 全部有界 cache 后重试，避免空闲内存永久困在局部层。

4.3 调度与定时器

单空闲核唤醒已经在第 3 章给出协议。实现层面，ready queue 用顺序容器保存公平次序，用集合保证 PID 唯一；producer 只有在成功插入后才可能领取 idle bit。该优化显著降低宿主无效 CPU，却没有改变 1,800 s crate 边界，因此被归类为资源效率修复，而不是主要吞吐来源。

周期 timer 原先每个 tick 都扫描包含历史 weak task 的全局 registry，QMP 多次采到同一热点。最终增加仅包含活动 interval timer owner 的弱引用索引；设置和撤销 timer 时更新索引，tick 只检

查小集合，并在投递信号前释放索引锁。该改动消除了采样热点，但 300 s 编译进度为 0%，所以只作为结构性优化保留。

4.4 ext4/VFS：正确性与吞吐的连续链条

4.4.1 大文件读取和命名空间事务

大型 `regular file` 读取显式遍历完整 `extent tree`，后部逻辑块不再依赖旧辅助映射。命名空间则以外层事务覆盖 `lookup` 与所有会改变目录项的操作。`rename` 目标替换与 `exchange` 的三步过程都位于一个事务内，`reader` 不会看到临时名字；`open-unlink` 立即移除目录项，但已打开 `inode` 引用继续维持数据寿命。

事务修复不把普通文件 I/O 串行化，也不取消 `generation`。`generation` 仍是防御性发布条件和诊断证据：若 `reader` 跨越 `writer` 代际，结果不能进入缓存。修复后的 300 s diagnostics 中 `positive/negative crossing` 均为 0。

4.4.2 稀疏大文件缓存与聚簇写回

小于 8 MiB 的 `regular file` 继续使用 `dense cache`，避免常见小文件访问承担额外树结构开销；大文件只为实际访问页面建立缓存。写回以 256 KiB cluster 处理 `dirty range`，连续物理 `extent` 的完整块最多合并 64 个请求，部分尾块仍执行读—改—写。`truncate` 会清理新 EOF 后的不完整块，共享文件按 `identity` 去重同步，保持 `fsync`、`rename` 和 `open-unlink` 语义。

该方案主要用于降低整文件缓存和逐块写回造成的长尾。它的完整运行比最快 `heap-cache` 样本略慢，因此本文把它称为稳定性候选，不将其包装成吞吐加速。

4.4.3 `lookup` 复用与有界工作集

同一次 `open` 的非 `tmpfs` 路径只做一次 `inode-kind` 查询，并在原调用生命周期内复用 `(inode, kind)`；`parent-symlink` 与 `readlink cache` 上限由 16K 增至 64K，达到上限时只淘汰一个确定性条目，不再清空整个工作集。`rename/unlink` 仍执行原有精确失效，`symlink` 跟随和 `tmpfs` 路由没有变化。

诊断中的三项热点按预测下降：`parent-symlink` 解析由约 49.80 s 降到 2.64 s，`final-symlink` 由 6.71 s 降到 3.81 s，普通 `ext4` 查询由 0.313 s 降到 0.008 s。由于设计只复用同一代际的已有结果，性能收益没有以放松一致性为代价。

4.5 ABI 与隐藏流程兼容

除了早期 IPC 与文件锁，最终流程还补齐了目录 FD 语义和同步信号。LoongArch64 评测日志中，完整 Rust 编译已经结束，后续 UEFI 准备最早失败于 `mkdir -p`；GNU `coreutils` 会保存目录 FD、切换工作目录，再通过 `fchdir` 恢复。共享 `syscall` 层因此按目录描述符解析和进程 `root/cwd` 状态实现该能力，并补齐 `fchmodat2` 的有限 `flags` 语义。无效 FD、非目录和不存在目标均返回对应错误，不伪造目录。

同步 `SIGILL` 通过既有 `signal frame` 投递，`rt_sigreturn` 恢复用户选择的新 PC。实现不识别 QEMU 名称或命令，只处理“已安装、未屏蔽的同步异常 `handler`”这一通用条件。独立 `nested-QEMU probe` 已从动态装载继续到 OpenSBI 并加载真实 wll_OS 内核，但由于使用派生镜像，它只属于 `capability-pass`。

第 5 章 优化实验与结果分析

5.1 阶段数据的正确读法

最初版本没有完整成功样本，只有 toolchain/minibuild 或 3,000—15,000 s 超时，因此不能计算“失败版本到成功版本”的严格加速比。进入双架构完整成功后，才具备可比较的阶段样本。本文同时保留三类数据：完整 production 时间用于结果判断，同配置 A/B 用于因果归因，诊断计数器用于解释热点。不同宿主、资源配置或 feature 的数字不直接相除。

阶段	RISC-V64	LoongArch64	主要变化	解释
早期基线	无成功 marker	无成功 marker	页表/ABI/生命周期缺口	不可计算加速比
Stage A 完整成功	1322.58 s	1103.14 s	所有权、高 arena、命名空间等正确性链	首次可比较成功样本
Stage B heap cache	800.55 s	660.71 s	per-CPU 小对象缓存	相对 Stage A 降低 39.47% / 40.11%
VFS lookup 候选	774.62 s	620.70 s	查询复用与有界缓存	相对其直接对照提升约 8%—11%
最终 Stage 17 树	686.50 s	558.27 s	MADV_DONTNEED 等累计修复	双架构公开流程 180/180

表 5-1 从正确性打通到最终候选的阶段结果

上表是一条阶段里程碑，不等同于每行只改变一个变量。Stage B 和 VFS 的隔离归因见后续两节；最终树还包含目录 ABI、资源拓扑、清页批处理和内存回收等累计修复，因此其总变化不能全部归给最后一项。

5.2 中央堆优化的单变量证据

Stage A 与 Stage B 的完整成功样本显示，RISC-V64 从 1322.58 s 降至 800.55 s，减少 522.03 s，即 39.47%；LoongArch64 从 1103.14 s 降至 660.71 s，减少 442.43 s，即 40.11%。两个架构方向一致，支持“中央小对象分配器是主要吞吐瓶颈”的判断。

同为 RISC-V64 diagnostics、SMP8、900 s 的计数器进一步建立机制证据：中央 heap 锁获取由 33,741,372 次降至 929,604 次，下降 97.24%；竞争由 5,004 次降至 7 次，下降 99.86%；小对象 cache 命中 20,032,390 次、miss 53,030 次，命中率 99.74%；192 个 QMP 样本中的中央 allocator 自旋簇由 35 个降为 0。编译事件由 129 增至 136，并在窗口内越过终点。

指标	Stage A	Stage B	变化
中央 heap 锁获取	33,741,372	929,604	-97.24%
中央 heap 锁竞争	5,004	7	-99.86%
小对象 cache 命中率	不适用	99.74%	新增局部复用
QMP 中央 allocator 自旋	35 / 192	0 / 192	热点消失
完整 production 时间（RV）	1322.58 s	800.55 s	-39.47%

表 5-2 per-CPU heap cache 的机制与结果

5.3 VFS 查询复用的同配置 A/B

VFS 候选在 QEMU 11.0.3、官方原始镜像、`-snapshot -m 8G -smp 8` 和 diagnostics 关闭的同配置 production 中验证。RISC-V64 的直接对照为 860.15—867.16 s，候选为 774.62 s，提升 9.94%—10.67%；LoongArch64 对照为 674.49—687.23 s，候选为 620.70 s，提升 7.97%—9.68%。两次候选都出现精确成功 marker，judge 为 180/180。

这一结果与诊断热点下降相互印证：减少的是重复 `ext4` 解析和整表清空，而不是编译器本身的用户态工作。`cache` 上限增加带来的内存成本被固定容量约束，淘汰策略虽不是完整 LRU，但消除了周期性冷启动悬崖，且不改变失效协议。

5.4 正确性和资源效率改动如何评价

命名空间事务、timer owner index 和单 idle CPU 唤醒都没有通过“显著缩短编译时间”的门槛。命名空间修复在 900 s 只从 129 增至 131 个编译事件，约 1.6%，但消除了 `ENOENT` 和 generation crossing；timer index 在 300 s 的 crate 进度为 0%，但 QMP 中全任务扫描热点消失；单核唤醒在 1,800 s 只快约 0.05%，却把 QEMU aggregate CPU 从约 718%—720% 降到 105%—108%，idle vCPU 自愿切换由约 52K—70K/s 降到约 95—131/s。

`MADV_DONTNEED` 的同机 LoongArch64 A/B 为 556.35 s 到 547.73 s，提升 1.55%。这个收益低于早期 5% 性能门槛，但它同时修复了“返回成功却没有回收”的 Linux 语义错误，使 135 次 jemalloc fallback 变为 0，因此作为正向正确性改动保留。

5.5 被证伪候选与取舍

为了避免只呈现成功实验，我们保留了对主要失败方向的记录。ready queue 线性去重替换、PTE/TLB batching、局部 VMA 合并、vacant-slot 策略和单独 timer index 都没有稳定越过生产进度边界；read-ahead 由 16 增至 32 的样本为 787.31 s，未优于 786.62 s 对照；second-chance cache 退化到 836.88 s；RISC-V activation token 使第 23 个 crate 慢 9.84%；munmap 局部 drain 只改善 1.16%。这些候选被回滚或降级为结构性修复，没有进入性能成果统计。

负结果改变了后续选择：当 `Vec<MapArea>` 上的多种局部技巧无法提升 production 时，我们停止继续调阈值，转而审计页级 owner 和虚拟地址容量；当 ASID 快路径出现 `SIGILL` 或生产变慢时，恢复保守边界，而不是用短诊断数据覆盖正式失败。

5.6 与本地 judge Linux 基线的比较

当前保存的 judge 配置使用 RISC-V64 1616 s、LoongArch64 1985 s 作为 Linux 基线。仅在该配置下，最终 686.50 s 对应时间降低 57.52%、基线/内核时间比 2.35 倍；最终 558.27 s 对应时间降低 71.88%、比值 3.56 倍。该表用于说明当前证据，不代替正式评测机重测的 Linux 基线，也不自行换算最终比赛分数。

架构	Linux 基线	wll_OS	时间降低	基线 / 本内核
RISC-V64	1616 s	686.50 s	57.52%	2.35×
LoongArch64	1985 s	558.27 s	71.88%	3.56×

表 5-3 最终时间与当前 judge 基线

第 6 章 测试体系与回归验证

6.1 验证矩阵

完整 BuildStorm 只能说明某一次综合负载通过，不能替代内部边界验证。每个保留候选至少经过静态检查、双架构构建、独立生命周期回归和官方工作负载四层门禁。production 与 diagnostics 分开构建，可以同时证明 feature 隔离没有破坏主路径。

验证	配置	结果	等级
release cfg	RV/LA × production/diagnostics	四项构建通过	capability-pass
SMP 生命周期	RV/LA, 8 GiB/8 CPU	双架构最终 pass cpus=8	capability-pass
用户内存	resident/high arena/COW/shared/file	双架构通过	capability-pass
页表与 TLB	隔离、ASID/root、复用、远端失效	双架构通过	capability-pass
文件系统	namespace/rename/open-unlink/fsync/truncate	双架构通过	capability-pass
目录 ABI	glibc/coreutils 风格目录 FD	双架构通过	capability-pass
内核堆	160 MiB stress + 320 MiB 大分配	双架构通过	capability-pass
LA 大资源启动	36 GiB/12 CPU	启动和 minibuild 通过	capability-pass
RV nested QEMU	派生 capability 镜像	OpenSBI 与真实内核加载	capability-pass
官方完整编译	未修改镜像, 8 GiB/8 CPU	双架构 marker + judge 180/180	official-pass

表 6-1 最终验证矩阵

6.2 独立回归覆盖的关键情形

SMP 回归不仅检查“八个 CPU 上线”，还要求八核都实际调度任务，并覆盖 affinity、外部唤醒、timer、等待队列和 heap stress。内存回归验证高地址 fault、部分 unmap、fork/COW、嵌套 fork 后只读 PTE、父进程再次写入时的物理分离，以及 shared mapping 保持同一物理页。地址空间退休测试还检查 exec 替换、旧 root 不再执行和 ASID 复用。

namespace 回归由七个固定 reader 与一个 writer 构成：writer 重复执行 64 次 negative-to-positive/create-unlink 转换，并验证同父/跨父 rename、open-unlink inode 寿命与最终缓存失效。MADV_DONTNEED 回归检查 discard 前后字节、翻译撤销、refault 清零以及 VMA 拓扑不变。目录 ABI 回归使用真实描述符切换 cwd，而不是只调用内部 helper。

6.3 正式成功条件

本地运行器对三个阶段使用完整条件：toolchain 必须出现 BUILDSTORM_TOOLCHAIN ok；minibuild 还必须出现 BUILDSTORM_MINIBUILD ok；complete 必须再出现精确 BUILDSTORM_COMPILE mode=multi ok=true。只出现 marker 前缀、fail 行、某个 crate 名或持续高 CPU 都不算完成。运行器只构建、启动、计时、保存和哈希，不向来宾注入 marker，也不修改 judge 语义。

6.4 生产与诊断隔离

production 关闭全部诊断计数和 QMP 辅助；diagnostics 仅使用固定原子、有限状态与单 CPU 周期汇总。源码审计确认不存在按测试名、路径、命令、输出、资源或时间选择行为；时钟、/proc/uptime 与在线 CPU 数据均来自内核真实状态。

第 7 章 可复现步骤与证据身份

7.1 权威输入

公开通过证据使用以下输入。复现前应重新计算哈希；任何一项变化都应被视为新的实验，而不是直接继承本文结果。

项目	身份
官方测例分支	final-2026
验证提交	b5ec6ef8497e1818cbdec3b54bb722f036e57972
guest 脚本 SHA-256	2f656a668076803fb465409374b6bcbb1fcbbc4f5c17a72b8ea4695668b9b33e
judge SHA-256	f9bc3c5c640217947775759b5b02aa4ceedfa76728d25d4f06d94ef5bc9d64dd
RISC-V64 镜像 SHA-256	d74e436522f5946ca17280a7a25f17dbb6604b71fe675bb8a021ce8e849b334c
LoongArch64 镜像 SHA-256	d1410544e677e11efb1c240be6ffb201c89d6de58c9675e73314a696e4cefdc5
QEMU	11.0.3
正式资源	-snapshot -m 8G -smp 8

表 7-1 官方输入与运行环境

7.2 更新并核验官方测例

以下命令在仓库根目录附近的固定工作区执行。路径可按本机调整，但 suite 提交、脚本、judge 和镜像哈希必须与记录一致。

```
git -C /srv/buildstorm/src/testsuits-for-oskernel fetch origin final-2026
git -C /srv/buildstorm/src/testsuits-for-oskernel checkout final-2026
git -C /srv/buildstorm/src/testsuits-for-oskernel rev-parse HEAD
git -C /srv/buildstorm/src/testsuits-for-oskernel status --short --branch

sha256sum /srv/buildstorm/src/testsuits-for-oskernel/scripts/buildstorm_testcode.sh
sha256sum /srv/buildstorm/src/testsuits-for-oskernel/judge/judge_buildstorm-glibc.py
sha256sum /srv/buildstorm/images/sdcard-rv-pub.img
sha256sum /srv/buildstorm/images/sdcard-la-pub.img
```

7.3 静态检查与双架构构建

```
git diff --check
cargo metadata --no-deps --format-version 1
cd os

WLL_HARNESS_GROUPS=buildstorm WLL_HARNESS_LIBC=glibc \
cargo +nightly-2025-01-18 build --locked --offline --release \
--target riscv64gc-unknown-none-elf

WLL_HARNESS_GROUPS=buildstorm WLL_HARNESS_LIBC=glibc \
cargo +nightly-2025-01-18 build --locked --offline --release \
--target riscv64gc-unknown-none-elf --features buildstorm-diagnostics

WLL_HARNESS_GROUPS=buildstorm WLL_HARNESS_LIBC=glibc \
cargo +nightly-2025-01-18 build --locked --offline --release \
--target loongarch64-unknown-none --no-default-features --features loongarch

WLL_HARNESS_GROUPS=buildstorm WLL_HARNESS_LIBC=glibc \
cargo +nightly-2025-01-18 build --locked --offline --release \
--target loongarch64-unknown-none --no-default-features \
--features loongarch,buildstorm-diagnostics
```

7.4 独立 SMP 回归

```
python3 scripts/run_smp_regression.py --arch riscv64 --timeout 180
python3 scripts/run_smp_regression.py --arch loongarch64 --timeout 180
```

通过条件应包含 namespace、目录 ABI、regular-file、resident/high-arena、user-memory、COW/shared/file、ASID/TLB、interval timer、heap stress 和最终八 CPU 成功行。不能只接受中间阶段的 `pass`。

7.5 官方完整运行与 judge

完整运行耗时较长，两个架构必须顺序执行。启动前先确认没有其他 QEMU 或 BuildStorm runner，避免宿主 CPU、内存和 I/O 相互污染。

```
pgrep -af 'qemu-system-(riscv64|loongarch64)|run_buildstorm.py' || true

python3 scripts/run_buildstorm.py --arch riscv64 \
  --image /srv/buildstorm/images/sdcard-rv-pub.img \
  --stage complete --timeout 15000 --memory 8G --smp 8

python3 scripts/run_buildstorm.py --arch loongarch64 \
  --image /srv/buildstorm/images/sdcard-la-pub.img \
  --stage complete --timeout 15000 --memory 8G --smp 8

python3 /srv/buildstorm/src/testsuits-for-oskernel/judge/judge_buildstorm-glibc.py \
  _tmp/buildstorm-riscv64-complete.log
python3 /srv/buildstorm/src/testsuits-for-oskernel/judge/judge_buildstorm-glibc.py \
  _tmp/buildstorm-loongarch64-complete.log
```

复现完成后，应同时保存原始串口、runner JSON、QEMU 完整参数、host elapsed、kernel/image/source/suite 哈希、构建日志及 judge stdout/stderr。成功判断以精确 marker、judge compile pass 和 provenance 完整为准。

7.6 证据索引

核心历史证据保存在 `docs/evidence/buildstorm-stage2/`。其中，双架构首次正式完成见 `20260814-stage2-buildstorm-official-completion.md`；per-CPU heap 归因见两个 `20260815-*-official-complete-stageb-percpu-heap-cache/` 目录；VFS 查询复用见 `20260815-vfs-lookup-reuse-validation.md`；大资源与同步信号见 `20260815-large-memory-smp-sigill-validation-cn.md`；最终 `MADV_DONTNEED` 与累计提交树见 `stage17-madvise-dontneed-official-20260816/README.md`。每个目录均应以其中的 raw serial 和 runner metadata 为准，而不是只引用汇总表。

第 8 章 AI 使用说明与人工核验

8.1 AI 参与的工作

本项目使用 OpenAI Codex / GPT-5 辅助处理以下任务：阅读源码、dirty diff、官方脚本和 judge，整理串口与聚合诊断；根据证据提出可证伪的根因模型和候选顺序；辅助实现通用内核修复、feature-gated diagnostics 与独立回归；编排双架构构建、单实例 QEMU、A/B 对照和哈希留存；最后将阶段材料重组为本文。

AI 的主要价值是缩短“日志—调用关系—可验证假设”之间的往返。例如，面对长时间高 CPU 停顿，AI 协助把宿主线程采样、QMP 返回地址和来宾计数器对齐，提出“广播 IPI 唤醒空闲核”的模型；面对 ENOENT，协助将 generation crossing 与 rename/unlink 锁边界对应起来。每个模型都必须能被单变量实验推翻，不能仅凭语言解释进入最终结论。

8.2 人工负责的判断与检查

开发者逐项审查所有 production 源码 diff，确认没有测试名、crate、路径、命令、输出、时间、CPU 数或预期 marker 特判；独立执行四种 release cfg、双架构 SMP 与生命周期回归；在未修改镜像上顺序启动 QEMU，检查原始串口并运行官方 judge；核对镜像、suite、脚本、judge、内核和源码差异哈希；对低于性能门槛、跨宿主或缺少 marker 的结果作降级说明。

所有候选的保留或回滚由可复核数据决定。AI 提出的 active-root、activation token、vacant-slot、read-ahead 和 cache 替换候选中，有的通过构建和 SMP，却在正式 workload 中失败或变慢；这些修改均被回滚，没有因为“实现已完成”就进入提交树。

8.3 未由 AI 或运行器改变的内容

AI 和本地运行器均未修改官方镜像、guest 脚本、judge、marker、guest 时间、/proc/uptime、在线 CPU 数或编译产物；没有预置被测二进制，也没有伪造输出。runner 只负责构建、启动、超时控制、日志保存、哈希和成功条件判定。diagnostics 不参与 production 运行，派生镜像得到的 nested-QEMU 结果只标为能力验证。

8.4 复核开发过程的最短路径

评审者可以按“最终 diff → 四 cfg 构建 → 双架构 SMP 串口 → 两份官方完整串口 → judge 输出 → 哈希”顺序独立复核。若需要验证性能归因，再查看 Stage A/B heap 计数与 VFS lookup A/B；若需要验证正确性边界，再查看 namespace crossing、MADV discard 和 nested-COW 回归。该顺序避免先阅读全部历史日志，也能重建本文的主要因果链。

第 9 章 限制、后续工作与总结

9.1 当前限制

- 最终评测机会同机重测 Linux 基线，本文的 2.35 倍和 3.56 倍只对应当前保存的 judge 配置。
- 公开 8 GiB/8 CPU 流程已正式通过；评测机出现过 16 GiB/8 CPU、36 GiB/12 CPU 和隐藏 UEFI/nested-QEMU 门。相关能力已建立独立回归，但新提交仍需以评测机结果确认。
- 页表更新仍以 4 KiB leaf 为主，当前只有特定撤销路径复用 2 MiB walk，尚未形成覆盖 map/protect/unmap 的统一批处理编辑 API。
- VMA 元数据仍以顺序容器组织。高 arena 解决了地址容量问题，局部算法降低了部分范围操作，但超大 VMA 集合的索引复杂度仍有演进空间。
- VFS parent/readlink cache 使用确定性有界淘汰而非统一 LRU。未来若合并缓存层，必须保留 namespace generation、精确失效和现有锁顺序。
- 本文结果来自 QEMU。VisionFive 2 与 Loongson 2K1000LA 的平台识别和架构边界已经保留，但不能把 QEMU 的 `official-pass` 直接表述为实板完成。

9.2 后续方向

后续优化应继续从完整 production 证据出发，而不是在已被削弱的候选上反复调参。内存方向可以研究不改变 owner 边界的 VMA 索引和统一页表批处理；文件系统方向可在现有事务失效协议上演进更精确的缓存替换与异步写回；调度方向应关注后期真正有用的两个 vCPU 之间的依赖和锁等待，而不是重新引入空闲核广播。

9.3 总结

BuildStorm 的跑通并非来自单点补丁，而是一条完整的工程链：先用最早故障边界修复地址空间和 ABI，使工具链真实工作；再用所有权、发布顺序和事务语义保证长时间、多进程、多核运行正确；随后通过隔离诊断把性能压力收敛到中央堆与 VFS 热路径，并用双架构 production A/B 验证收益；最后以独立生命周期回归、未修改官方输入、精确 marker、judge 和哈希封闭证据。

最终设计的核心不是记住若干函数，而是把责任放回正确层次：VMA 描述策略，驻留集拥有实际页，页表表达翻译，架构层维护 TLB；调度器先发布任务再唤醒；文件系统在同一命名空间事务内解析、修改与失效；缓存只优化其责任单元，不取得上层对象所有权。正是这些可以被解释、测试和复现的不变量，使 wll_OS 从无法执行第一条脚本命令，演进为 RISC-V64 与 LoongArch64 均能稳定完成官方全量编译的内核。